
Trabajo Práctico Integrador

Gestión de Datos de Países en Python

Filtros, Ordenamientos y Estadísticas

Materia: Programación 1

Institución: UTN — Tecnicatura Universitaria en Programación

Modalidad: A Distancia

Integrantes:

Alejandro Fernández · Gallardo Josue

Fecha: Junio 2026

2.1 Estructura del sistema	8
2.2 Diagrama de flujo	9
2.3 Capturas de Pantalla	11
3. Dificultades y Conclusiones	12
4. Bibliografía / Webgrafía	13
5. Enlaces al repositorio y video	14

1. MARCO TEÓRICO

1.1 Listas

Una **lista** en Python es una estructura de datos ordenada y mutable que permite almacenar múltiples elementos de cualquier tipo bajo un mismo nombre de variable. Se definen con corchetes `[ ]` y sus elementos se acceden mediante índices numéricos comenzando desde 0. En este proyecto, la variable global **países** es una lista de diccionarios que actúa como la base de datos en memoria del sistema.

```
# Lista vacía que almacenará todos los países
países = []

# Agregar un elemento
países.append({"nombre": "Argentina", "poblacion": 45376763})

# Acceder al primer elemento
print(países[0]["nombre"]) # Argentina

# Longitud
print(len(países)) # 1
```

Las listas admiten `append()` e iteración con `for`. Son fundamentales para colecciones dinámicas.

1.2 Diccionarios

Un **diccionario** es una estructura de datos que almacena pares **clave: valor**. A diferencia de las listas, el acceso a los elementos se realiza por clave (string o número), no por posición. En este proyecto, cada país es representado como un diccionario con las claves *nombre*, *poblacion*, *superficie* y *continente*.

```
# Cada país es un diccionario
pais = {
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
    "continente": "América"
}

# Acceder a un campo
print(pais["nombre"]) # Argentina

# Modificar un campo
```

```

pais["poblacion"] = 46000000

# Iterar claves y valores
for clave, valor in pais.items():
    print(f"{clave}: {valor}")

```

Los diccionarios son ideales para modelar entidades con múltiples atributos heterogéneos. También se utilizan en este sistema para contar países por continente (*continentes = {}*), donde la clave es el nombre del continente y el valor es el conteo.

1.3 Funciones

Una **función** es un bloque de código reutilizable que realiza una tarea específica. Se definen con la palabra clave **def** y pueden recibir parámetros y retornar valores. El principio de **una función = una responsabilidad** garantiza código modular, legible y fácil de mantener.

```

def validar_numero(mensaje):
    """Solicita un número entero positivo al usuario."""
    while True:
        try:
            numero = int(input(mensaje))
            if numero < 0:
                print("Error: debe ser positivo.")
            else:
                return numero
        except ValueError:
            print("Error: ingrese un número válido.")

```

En este proyecto se utilizan funciones para cada funcionalidad: *cargar_paises()*, *agregar_pais()*, *buscar_pais()*, *filtrar_continente()*, *ordenar_paises()*, *mostrar_estadisticas()*, etc. Cada función tiene una única responsabilidad, lo que facilita la depuración y el testing.

1.4 Condicionales

Las **estructuras condicionales** permiten ejecutar diferentes bloques de código según si una condición es verdadera o falsa. Python utiliza **if / elif / else**. En este sistema se emplean extensamente para validar entradas, controlar el menú principal, verificar si un país existe antes de actualizarlo y filtrar resultados.

```
# Control del menú principal
if opcion == "1":
    agregar_pais()
elif opcion == "2":
    actualizar_pais()
elif opcion == "3":
    buscar_pais()
elif opcion == "0":
    print("Programa finalizado.")
    break
else:
    print("Opción inválida.")

# Validación en bucle
if texto == "":
    print("Error: no puede estar vacío.")
elif not texto.replace(" ", "").isalpha():
    print("Error: solo se permiten letras.")
else:
    return texto
```

Las condicionales también se combinan con bucles **while True** en las funciones de validación, garantizando que el programa no avance hasta recibir datos correctos.

1.5 Ordenamientos

Ordenamiento por método de burbuja con for anidados. En `ordenar_paises()` se copia países a ordenados. Se compara por nombre (`.lower()`), población o superficie, ascendente (A) o descendente (D).

```
for i in range(cantidad):
    for j in range(0, cantidad - i - 1):
        if ordenados[j]["poblacion"] > ordenados[j + 1]["poblacion"]:
            auxiliar = ordenados[j]
            ordenados[j] = ordenados[j + 1]
            ordenados[j + 1] = auxiliar
```

1.6 Estadísticas Básicas

? Mayor y menor población recorriendo con for.

? Promedios con acumuladores total_poblacion y total_superficie.

? Conteo por continente con diccionario y if/else.

```
mayor = paises[0]
for pais in paises:
    if pais["poblacion"] > mayor["poblacion"]:
        mayor = pais
total_poblacion = total_poblacion + pais["poblacion"]
```

1.7 Archivos CSV

Persistencia en `países.csv`. Ejemplo: Argentina,45376763,2780400,América

`archivo.readline()` # salta encabezado

`try/except ValueError` por fila inválida

`guardar_pais_csv()` en `append`; `guardar_todos_los_paises_csv()` en actualización.

2. DECISIONES TÉCNICAS Y ARQUITECTURA

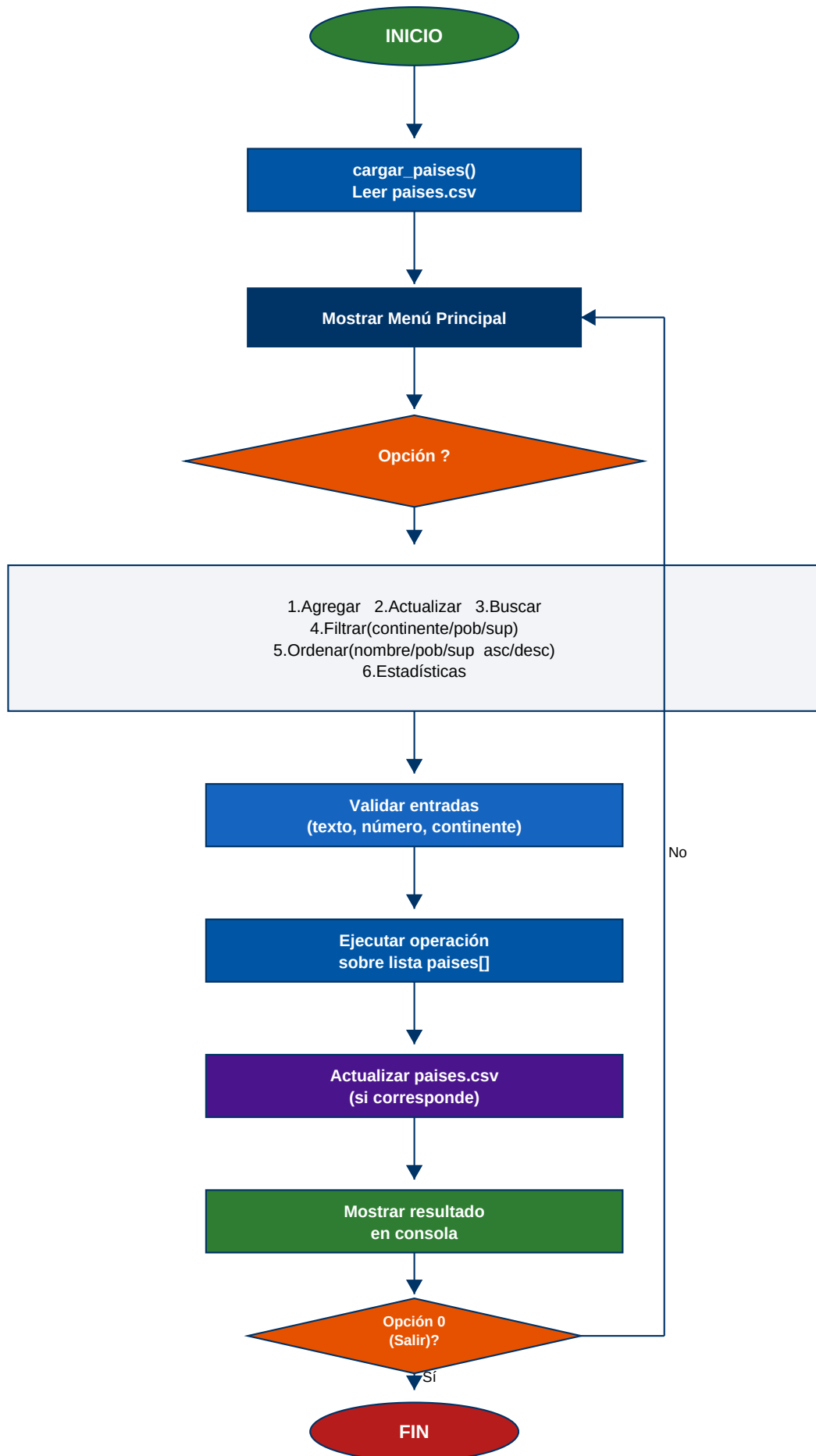
2.1 Estructura del Sistema

El sistema está organizado en módulos funcionales dentro de un único archivo Python. Cada responsabilidad tiene su propia función, siguiendo el principio de una función por responsabilidad. Cada módulo de datos usa una lista global de diccionarios sincronizada con el archivo CSV.

Módulo	Funciones principales	Descripción
Carga/Guardado	cargar_paises() guardar_pais_csv() guardar_todos_los_paises_csv() cargar_paises_csv()	Persistencia en archivo CSV
Validaciones	validar_texto() validar_numero() validar_continente()	Control de entradas del usuario
CRUD	agregar_pais() actualizar_pais() buscar_pais()	Operaciones sobre los datos
Filtros	filtrar_continente() filtrar_poblacion() filtrar_superficie()	Subconjuntos del dataset
Ordenamiento	ordenar_paises()	Burbuja con for anidados o
Estadísticas	mostrar_estadisticas()	For y acumuladores
Menú	mostrar_menu() menu_filtros()	Interfaz de usuario en consola

2.2 Diagrama de Flujo — Operaciones Principales

A continuación se presenta el diagrama de flujo del programa, desde el inicio hasta la salida, pasando por cada opción del menú principal.



Leyenda del diagrama

Forma	Significado
Óvalo verde/rojo	Inicio / Fin del programa
Rectángulo azul	Proceso / Operación
Rectángulo gris	Detalle de suboperaciones
Rombo naranja	Decisión (bifurcación del flujo)

2.3 Capturas de Pantalla

Incluir capturas de consola al ejecutar Integrador.py:

- ? Carga del CSV y menú principal
- ? Agregar país con validaciones
- ? Buscar por coincidencia parcial
- ? Filtrar por continente o rango
- ? Ordenar ascendente/descendente
- ? Estadísticas completas

[Reemplazar este apartado con las imágenes reales antes de entregar.]

3. DIFICULTADES Y CONCLUSIONES

3.1 Dificultades

- ? Continentes con/sin tilde: solución con `CONTINENTES_MAPA`.
- ? Filas inválidas en CSV: `try/except ValueError` por línea.
- ? Rangos de filtros: validación `minimo <= maximo`.
- ? Duplicados al agregar: comparación con `.lower()`.
- ? Orden alfabético: `.lower()` en comparaciones del burbuja.

3.2 Conclusiones

Se aplicaron listas, diccionarios, funciones y archivos CSV para construir un sistema modular en consola. Las validaciones y mensajes de error mejoran la robustez. El trabajo en equipo permitió dividir código, informe y pruebas.

- [2] Python Software Foundation. (2024). Data Structures. <https://docs.python.org/3/tutorial/datastructures.html>
- [3] Python Software Foundation. (2024). Input and Output. <https://docs.python.org/3/tutorial/inputoutput.html>
- [4] Python Software Foundation. (2024). Control Flow. <https://docs.python.org/3/tutorial/controlflow.html>
- [5] Lutz, M. (2013). Learning Python. O'Reilly Media.
- [6] Real Python. (2024). CSV Files in Python. <https://realpython.com/python-csv/>

5. ENLACES AL REPOSITORIO Y VIDEO

Repositorio GitHub:

<https://github.com/alevanfof/appPaísesIntegradorProgl2026>

Video demostrativo (10-15 min, acceso público):

[PENDIENTE ? URL de YouTube o Drive]

Informe PDF en la raíz del repositorio.